

Politecnico di Torino

Mechatronics Engineering

A.a. 2025/2026

Graduation Session Mese Anno

Titolo della tesi

Eventuale sottotitolo

Relatori:

Name Surname

Name Surname

Candidati:

Emanuele Pruiti

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VII

1 Introduction 1
- 1.1 Context and motivation . . . 1
- 1.2 Objectives of the thesis . . . 1
- 1.3 Organization of the work . . . 2

2 Background and Problem Formulation 3
- 2.1 Agricultural autonomous navigation . . . 3
  - 2.1.1 Navigation tasks and requirements . . . 3
  - 2.1.2 Environmental challenges . . . 3
- 2.2 Vision-based local guidance . . . 4
  - 2.2.1 Sensing modalities . . . 4
  - 2.2.2 Image-space reference extraction . . . 4
- 2.3 The segmentation-to-control pipeline . . . 4
  - 2.3.1 Perception stage . . . 4
  - 2.3.2 Geometric extraction stage . . . 5
  - 2.3.3 Control integration . . . 5

3 State of the Art 6
- 3.1 Classical image-processing methods . . . 6
- 3.2 Learning-based segmentation for agricultural navigation . . . 6
  - 3.2.1 Semantic segmentation approaches . . . 6
  - 3.2.2 Training data and generalization . . . 7
- 3.3 Semantic segmentation architectures . . . 7
  - 3.3.1 FCN, DeepLabV3, and mobile architectures . . . 7
  - 3.3.2 Transfer learning and pretraining . . . 7
- 3.4 Dataset preparation and model-assisted annotation . . . 8
  - 3.4.1 Manual annotation challenges . . . 8
  - 3.4.2 SAM and foundation models for segmentation . . . 8
- 3.5 Geometric extraction methods . . . 8

3.5.1 Line fitting and crop-edge detection . . . 8
3.5.2 Column-wise histogram methods . . . 9
3.5.3 Horizontal strips and adaptive multi-ROI methods . . . 9
3.5.4 Triangle Scan Method . . . 9
3.5.5 Comparison of geometric representations . . . 9
3.6 System integration and experimental validation . . . 11
3.6.1 Perception and control integration . . . 11
3.6.2 Scope of reported evaluations . . . 11
3.7 Positioning of this thesis . . . 11

4 Deep Learning Architectures for Semantic Segmentation 13
4.1 Problem formulation and encoder-decoder principles . . . 13
4.2 Fully Convolutional Networks (FCN) . . . 14
4.2.1 Fully convolutional transformation . . . 14
4.2.2 Skip architecture and upsampling . . . 15
4.3 DeepLabV3 and atrous convolutions . . . 15
4.3.1 Atrous convolution formulation . . . 15
4.3.2 Atrous Spatial Pyramid Pooling (ASPP) . . . 16
4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head . . . 17
4.4.1 Depthwise separable convolutions . . . 17
4.4.2 Inverted residual blocks with linear bottlenecks . . . 18
4.4.3 Squeeze-and-Excitation (SE) channel attention . . . 18
4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid . . . 19
4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture . . . 19
4.5 Segment Anything Model 3 (SAM 3) for dataset annotation . . . 20
4.5.1 Architecture overview . . . 20
4.5.2 Operational distinction from onboard models . . . 21
4.6 Summary of architectural characteristics . . . 21

5 Dataset Preparation and Comparative Model Training 23
5.1 Image acquisition and class definition . . . 24
5.1.1 Image sources and acquisition conditions . . . 24
5.1.2 Semantic classes and labeling targets . . . 24
5.2 Dataset preparation . . . 24
5.2.1 Lettuce dataset with manual annotation . . . 24
5.2.2 SAM 3-assisted annotation for multi-crop datasets . . . 24
5.2.3 Dataset partitioning . . . 25
5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP . . . 26
5.3.1 Model selection and pretrained weights . . . 26
5.3.2 Training configuration . . . 27
5.3.3 Evaluation metrics and results . . . 27

5.3.4 Selection of LR-ASPP 28
5.4 Supervision strategy comparison 28
5.4.1 Training configurations and evaluation targets 28
5.4.2 Evaluation results 29
5.4.3 Selection of binary path configuration 29
5.5 Model export and inference configuration 30
5.5.1 ONNX export 30
5.5.2 Inference inputs and outputs 30

6 Geometric Extraction of the Local Navigation Reference 31
6.1 Input representation and edge-line models 31
6.2 Region of interest and mask preparation 32
6.3 Horizontal strips and candidate intervals 33
6.3.1 Strip construction and column projection 33
6.3.2 Method-specific local support 34
6.4 Initialization and bottom-up association 34
6.4.1 Method 3: lowest supported seed 34
6.4.2 Method 4: edge-support seed and gap-limited association . . 35
6.5 Separate robust fitting of the corridor edges 36
6.5.1 Excluding image-boundary observations 36
6.5.2 Residual-based reweighting 36
6.6 Quality indicators, validation, and image-space output 37
6.6.1 Method 3: centerline support after rasterization 37
6.6.2 Method 4: geometric checks and a composite score 37
6.6.3 Interpretation of status and confidence 38
6.7 Default configurations and comparison 39
6.8 Effect of strip height and number 40

7 Robot Platform and Integration 43
7.1 Robot hardware and sensors 43
7.1.1 Robot platform 43
7.1.2 Camera configuration 43
7.1.3 Computing platform 43
7.2 Software architecture 43
7.2.1 ROS integration 43
7.2.2 Perception pipeline implementation 44
7.2.3 Control interface 44
7.3 Deployment configuration 44
7.3.1 Model export to ONNX 44
7.3.2 Real-time inference 44
7.3.3 Timing and latency 44

8 Experimental Validation 45
8.1 Offline evaluation protocol . . . 45
8.1.1 Segmentation metrics . . . 45
8.1.2 Geometric reference accuracy . . . 46
8.2 Field validation . . . 46
8.2.1 Test conditions . . . 46
8.2.2 Closed-loop navigation results . . . 46
8.2.3 Qualitative observations . . . 46
8.3 Discussion . . . 46
8.3.1 Segmentation performance . . . 46
8.3.2 Geometric extraction performance . . . 46
8.3.3 Limitations . . . 47

9 Conclusions and Future Work 48
9.1 Main contributions . . . 48
9.2 Conclusions on segmentation and annotation . . . 49
9.3 Conclusions on geometric extraction . . . 49
9.4 Limitations . . . 49
9.5 Future work . . . 50
9.5.1 Dataset and annotation . . . 50
9.5.2 Segmentation and model deployment . . . 50
9.5.3 Geometric extraction . . . 51
9.5.4 System integration and field validation . . . 51
9.6 Closing remarks . . . 51

Bibliography 53

List of Figures

Chapter 1

Introduction

1.1 Context and motivation

Autonomous navigation in agricultural environments requires robots to follow crop rows reliably under variable field conditions. Vision-based guidance uses cameras to identify the navigable path between rows, but lighting changes, occlusions, weeds, and plant growth affect image appearance. Manual pixel-level annotation for training segmentation models is time-consuming and must cover multiple crops and conditions.

This thesis addresses these challenges through a combination of learning-based semantic segmentation and geometric reference extraction. It investigates model-assisted annotation to reduce manual labeling effort, compares segmentation architectures suitable for real-time robot perception, and develops geometric methods that convert corridor masks into local navigation references.

1.2 Objectives of the thesis

The main objectives of this work are:

- Compare manual and SAM 3-assisted annotation strategies for agricultural segmentation datasets spanning lettuce, tomato, and cucumber.
- Select a segmentation architecture balancing accuracy and inference speed for robotic deployment, through offline evaluation of FCN, DeepLabV3, and LR-ASPP on manually annotated lettuce images.
- Evaluate the effect of output label configurations (four-class, three-class, and binary path) on path segmentation accuracy.

- Implement and compare two edge-based geometric extractors that convert segmentation masks into local image-space navigation references.
- Integrate the selected perception and geometric components on a robot platform for field validation.

1.3 Organization of the work

The thesis is organized as follows:

Chapter 2 introduces the agricultural navigation problem, the role of vision-based local guidance, and the segmentation-to-control pipeline structure.

Chapter 3 reviews vision-based methods for agricultural row navigation, covering classical and learning-based segmentation approaches, geometric extraction techniques, and experimental validation in related work.

Chapter 4 details the mathematical formulations and designs of the deep learning architectures evaluated in this work, focusing on FCN, DeepLabV3, MobileNetV3 with the LR-ASPP head, and SAM 3.

Chapter 5 describes dataset preparation with manual and SAM 3-assisted annotation, empirical comparisons among the three architectures on lettuce, and evaluations of supervision strategies across label configurations.

Chapter 6 presents the two geometric extractors that convert corridor masks into navigation references through horizontal-strip interval selection and independent edge fitting.

Chapter 7 describes the robot hardware, camera configuration, software integration using ROS, and the deployment of the trained segmentation model.

Chapter 8 reports offline and field validation results for segmentation accuracy, geometric reference extraction, and closed-loop navigation performance.

Chapter 9 summarizes the main findings, discusses limitations, and identifies directions for future work.

Chapter 2

Background and Problem Formulation

This chapter establishes the agricultural navigation context and formulates the vision-based local guidance problem addressed in this thesis.

2.1 Agricultural autonomous navigation

2.1.1 Navigation tasks and requirements

Agricultural robots perform tasks such as monitoring, spraying, harvesting, and weeding in structured crop environments. Navigation between crop rows requires the robot to follow a path aligned with the row structure while avoiding plants and obstacles. The guidance reference must be computed from local sensing, as global GNSS positioning may be obstructed by crop canopies or greenhouse structures.

Vision-based methods identify crop rows, inter-row paths, or crop boundaries from camera images and convert these observations into steering commands. The task is local: the robot maintains its position relative to the visible row structure, not a global map.

2.1.2 Environmental challenges

Field conditions vary with illumination, shadows, weather, growth stage, and weed presence. Plants may occlude parts of the path, and gaps or irregular spacing disrupt row structure. These changes affect both the appearance of image features and the reliability of geometric references extracted from them.

Segmentation models trained on one set of conditions must generalize to new lighting, crops, or row configurations. Geometric extractors must handle incomplete

or ambiguous corridor observations without assuming perfect segmentation.

2.2 Vision-based local guidance

2.2.1 Sensing modalities

Camera-based perception provides appearance information for distinguishing plants, soil, and paths. Monocular RGB cameras are simpler and less expensive than LiDAR or stereo systems, but do not directly measure depth. RGB-D cameras combine appearance and depth; their suitability depends on field conditions and cost constraints.

This thesis uses monocular RGB sensing for local guidance. It does not reconstruct three-dimensional field structure or maintain a global map.

2.2.2 Image-space reference extraction

The guidance pipeline converts image observations into a reference suitable for control. Depending on the method, this may be a horizontal position, a fitted line, or a curve. Image-space references describe the path geometry in pixel coordinates; they must be interpreted by the controller to generate velocity commands.

Straight-line models assume locally linear boundaries and produce simple references for proportional control. They do not capture distant curves or complex row patterns, but are computationally efficient and suitable for short prediction horizons.

2.3 The segmentation-to-control pipeline

2.3.1 Perception stage

The perception stage assigns semantic labels to image pixels. For inter-row navigation, relevant classes may include plants, soil, path, background, and covering material. The choice of output classes affects what the geometric stage can extract: a vegetation mask differs from a path mask, and a crop-row label differs from a soil corridor.

Segmentation models are trained on annotated datasets. Transfer learning from pretrained weights can reduce training time and improve generalization, but the output layers must be adapted to the target task.

2.3.2 Geometric extraction stage

The geometric stage reduces the segmentation mask to a navigation reference. Methods differ in their input representation, search strategy, and fitted model. Some aggregate mask pixels by image column; others select local evidence from horizontal strips or search for lines within a bounded region.

The output must be robust to segmentation errors, gaps, and occlusions. Validation checks can reject references that fail quality thresholds, but the controller must handle cases where no valid reference is available.

2.3.3 Control integration

The controller interprets the navigation reference to generate linear and angular velocity commands. Proportional control uses the lateral displacement and heading error of the reference relative to the image center. Temporal filtering can smooth references across frames, but introduces lag.

Closed-loop evaluation measures the robot's ability to follow rows under field conditions. Segmentation accuracy and geometric reference quality are necessary but not sufficient: the complete system must maintain corridor alignment over extended distances and variable conditions.

Chapter 3

State of the Art

This chapter reviews vision-based methods for agricultural row navigation, focusing on the conversion of image observations into a local guidance reference. It covers visual perception, geometric extraction, and the integration of these stages with robot control. Dataset preparation and the implementation used in this thesis are described in Chapters 5 and 6.

3.1 Classical image-processing methods

Classical pipelines combine image filtering, thresholding, and application-specific geometric rules. Mendez et al. convert vineyard images to grayscale, apply Gaussian filtering and Otsu thresholding, select the largest blob, and use column pixel counts to estimate the row center [1, Sections 3.4–4]. Such methods can be effective in the conditions for which their parameters were selected, but changes in illumination, weeds, and crop appearance can require retuning [2, Section 2].

3.2 Learning-based segmentation for agricultural navigation

3.2.1 Semantic segmentation approaches

Semantic segmentation assigns a class to each pixel, but the label definition determines what the geometric stage can recover. De Silva et al. train U-Net to predict narrow crop-row masks in sugar-beet images; the labels represent row structure rather than individual plant pixels or traversable soil [3, Sections 3.3 and 4.1]. Navone et al. instead predict a binary vegetation mask from RGB images using a MobileNetV3 backbone and an LR-ASPP head, then combine that output with depth for reference extraction [4, Sections 3.3 and 4.1].

The distinction matters for this work: its target is the soil region between crop rows, not a crop-row line or a vegetation mask. A segmentation model therefore supplies a candidate region; the geometric stage must still select the corridor and construct the guidance reference.

3.2.2 Training data and generalization

Pixel-level annotation is time-consuming, and agricultural datasets must represent changes in crop, growth stage, and lighting. De Silva et al. include 11 field variations in their dataset [3, Section 3]. Navone et al. initialize their segmentation model with ImageNet weights and train using synthetic AgriSeg images supplemented by 100 real crop images [4, Section 4.1, Table 1]. Pretraining and the choice of training images are separate design choices.

3.3 Semantic segmentation architectures

3.3.1 FCN, DeepLabV3, and mobile architectures

Fully Convolutional Networks (FCN) replace fully connected layers with convolutions to produce dense predictions. DeepLabV3 introduces atrous convolution and atrous spatial pyramid pooling to capture multi-scale context. LR-ASPP (Lite Reduced Atrous Spatial Pyramid Pooling) reduces computational cost for mobile deployment while maintaining segmentation accuracy.

The choice of architecture affects both offline accuracy and real-time inference speed. Models with deeper backbones or larger receptive fields may achieve higher IoU but require more computation. For robotic applications, the trade-off between accuracy and latency determines which architecture is deployable.

3.3.2 Transfer learning and pretraining

Transfer learning initializes network weights from models pretrained on large-scale datasets such as ImageNet or COCO. The backbone extracts general visual features; the segmentation head is adapted to the target task by replacing output layers with new convolutions matching the desired class count.

Pretrained weights reduce training time and improve generalization, especially when target datasets are small. However, the extent of pretraining differs: some implementations retain pretrained segmentation heads and replace only the final classifier, while others initialize only the backbone and train the head from scratch.

3.4 Dataset preparation and model-assisted annotation

3.4.1 Manual annotation challenges

Manual pixel-level annotation requires annotators to label every pixel in training images. For agricultural datasets spanning multiple crops, growth stages, and lighting conditions, this process is time-consuming and expensive. Annotation consistency depends on annotator interpretation of ambiguous boundaries, particularly where plants overlap soil or covering material.

3.4.2 SAM and foundation models for segmentation

Segment Anything Model 3 (SAM 3) supports text- and image-prompted concept segmentation. Its training data were assembled using a data engine with human and automated quality checks [5, Sections 2 and 4]. Foundation models like SAM 3 can assist annotation by generating masks from text prompts, reducing manual effort.

In this thesis, SAM 3 is used to prepare labels offline, not as the network running during robot navigation. The quality of these masks and the time required to review or correct them are evaluated separately in Chapter 5.

3.5 Geometric extraction methods

The geometric stage reduces a mask to a quantity the controller can use. Depending on the representation and method, the output may be a horizontal image position, a straight line, a crop-edge estimate, or a curve. These outputs describe different tasks and coordinate systems.

3.5.1 Line fitting and crop-edge detection

Chen et al. estimate a path line for greenhouse cucumber rows from navigation points selected in plant–soil segmentation. Their prediction-point Hough transform uses a least-squares estimate to restrict the line search and is compared with traditional Hough and least-squares fitting [6, Sections 2–3].

Kneip et al. detect crop–ground transitions in scanlines of a stereo elevation map, then fit a linear crop-edge model using a Huber M-estimator and iteratively reweighted least squares. Their output is a crop boundary in ground-plane coordinates, not an image-space estimate of the inter-row path [2, Sections 6–7].

3.5.2 Column-wise histogram methods

Histogram methods aggregate mask pixels by image column. Mendez et al. estimate the row center from the five columns with the largest pixel counts in a thresholded background region [1, Section 3.4.3]. Navone et al. use a different input: a temporally combined vegetation mask refined with depth. SegMin smooths its column histogram and selects the minimum; SegMinD weights the vegetation mask by normalized inverse depth before calculating the histogram [4, Section 3]. In both cases the output is a horizontal position, not a full path line.

3.5.3 Horizontal strips and adaptive multi-ROI methods

Li et al. combine a Faster-U-net inter-row-path mask with an adaptive multi-ROI procedure. Horizontal regions supply ordered local points, which are used as control points for a multisegment cubic B-spline. Their experiments use 40-pixel search regions and discuss the trade-off between the number of points and fitting behavior [7, Section 2.2.3, Figure 9].

This is the closest geometric comparison for the present work because both pipelines select local evidence from horizontal image regions. The representation and fitted model differ: Li et al. use an inter-row-path mask and a spline, whereas this thesis estimates a local straight reference from the soil corridor between parallel crop rows.

3.5.4 Triangle Scan Method

De Silva et al. propose the Triangle Scan Method (TSM) for crop-row masks. It first scans an upper horizontal band to find an anchor column, using a preset anchor if the scan fails a threshold. Candidate lines from that anchor to positions along the base of a triangular region are then evaluated; the line with the greatest mask support defines the central crop row. Its two endpoints provide position and angular errors for visual servoing [3, Section 4.2, Algorithm 1].

TSM is not the same as extracting one point from each of several horizontal strips: the upper scan establishes an anchor for a bounded line search. It also operates on crop-row masks, so its output is not directly equivalent to a reference extracted from a soil mask.

3.5.5 Comparison of geometric representations

Table 3.1 summarizes the input representation and geometric output of the methods discussed above.

The methods are not directly ranked by this comparison: their inputs and outputs differ. Li et al. provide the closest structural precedent for strip-based

Table 3.1: Perceptual inputs and geometric references in selected agricultural navigation studies.

Study	Perceptual input	Geometric operation	Output
Chen et al. [6]	Plant–soil segmentation and navigation points	Restricted Hough search around a predicted point	Image-space path line
Kneip et al. [2]	Stereo elevation map	Scanline transitions and robust linear fit	Ground-plane crop edge
Mendez et al. [1]	Thresholded background region	Column counts and mean of the five largest	Horizontal row-center position
Navone et al. [4]	Vegetation mask with depth	Smoothed column histogram; inverse-depth weighting in SegMinD	Histogram-minimum position
Li et al. [7]	Predicted inter-row-path mask	Adaptive horizontal regions and cubic B-spline	Curved image-space path
De Silva et al. [3]	Predicted crop-row mask	Upper anchor and triangular line search	Central crop-row line

point selection, while TSM illustrates a line search for a crop-row representation. Neither method uses the same soil-corridor mask and straight-line model as the pipeline developed in this thesis.

3.6 System integration and experimental validation

3.6.1 Perception and control integration

Navigation systems use the extracted reference to generate motion commands. Navone et al. convert the displacement of their histogram reference into linear and angular velocity commands and smooth them with an Exponential Moving Average [4, Section 3.4]. Mendez et al. use proportional control based on row-center displacement and the distribution of background pixels [1, Section 3.5]. De Silva et al. combine the central row's angular and positional errors in a proportional visual-servoing controller for simulation [3, Section 4.3].

3.6.2 Scope of reported evaluations

The studies evaluate different stages of navigation. Kneip et al. assess crop-edge detection on recorded harvesting data [2, Section 8]; Chen et al. report angular errors and processing times for image-based path fitting [6, Sections 3.6, Tables 6–7]. De Silva et al. evaluate row detection on real images and visual servoing in simulation, with preliminary real-world row following also reported [3, Sections 5.2–5.3 and 6, Table 5]. Mendez et al. report closed-loop vineyard trials, measuring centering error from the distances to the two sides of the row [1, Sections 3.6 and 4]. Navone et al. test in simulation and crop fields, using LiDAR and attitude measurements to assess displacement and heading [4, Sections 4.3–4.4].

Segmentation IoU, image-space line error, field tracking error, and processing time measure different parts of the system. Results from these studies therefore provide context, not a common performance benchmark.

3.7 Positioning of this thesis

This thesis studies a camera-based pipeline that segments plants and the soil corridor, selects the relevant inter-row region, and estimates a local guidance line for navigation between parallel crop rows. Its geometric stage uses horizontal strips to select candidate intervals and fits the two corridor boundaries separately. This shares the local-selection structure of Li et al.'s multi-ROI method, but uses a straight image-space reference rather than a spline. The experiments in the

following chapters evaluate the segmentation models, geometric extractors, and integrated robot at their respective stages.

Chapter 4

Deep Learning Architectures for Semantic Segmentation

Semantic segmentation in agricultural robotics consists in assigning a predefined category label to every pixel of an acquired image. For autonomous row navigation, the perception system must distinguish navigable soil corridors from surrounding crop canopies, weeds, and field boundaries under variable outdoor illumination.

This chapter presents the mathematical formulation and architectural design of the deep learning models evaluated in this work. First, the segmentation task and encoder-decoder principles are defined. Second, Fully Convolutional Networks (FCN) and DeepLabV3 are detailed with emphasis on atrous convolutions and multi-scale context aggregation. Third, MobileNetV3 and the Lite Reduced Atrous Spatial Pyramid Pooling (LR-ASPP) head are analyzed, detailing the depthwise separable convolutions and channel attention mechanisms that reduce computational complexity for edge computing. Finally, the Segment Anything Model 3 (SAM 3) is described as an offline annotation engine for training label generation.

4.1 Problem formulation and encoder-decoder principles

Let $I \in \mathbb{R}^{H \times W \times 3}$ denote an input color image with height H and width W. Semantic segmentation aims to estimate a dense label map $Y \in \{1, \ldots, C\}^{H \times W}$, where each pixel (i, j) is assigned to one of C mutually exclusive semantic classes. Deep neural networks approximate this mapping through a parameterized function $f_\theta(I)$ that outputs an unnormalized score tensor $Z \in \mathbb{R}^{H \times W \times C}$. The posterior probability of

class c at pixel (i, j) is obtained via the spatial softmax function:

$$p_{i,j,c} = \frac{\exp(Z_{i,j,c})}{\sum_{k=1}^{C} \exp(Z_{i,j,k})}. \tag{4.1}$$

During supervised training with ground-truth one-hot masks $Y^* \in \{0,1\}^{H \times W \times C}$, parameters θ are optimized by minimizing the multi-class cross-entropy loss:

$$\mathcal{L}_{\text{CE}}(Y^*, p) = -\frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \sum_{c=1}^{C} Y^*_{i,j,c} \log(p_{i,j,c}). \tag{4.2}$$

Standard convolutional architectures for image classification progressively reduce spatial dimensions via strided convolutions and max-pooling layers. While spatial reduction increases the receptive field and aggregates contextual information, it discards fine-grained boundary positions. Semantic segmentation architectures solve this trade-off using an encoder-decoder topology:

- **Encoder (Backbone):** A sequence of convolutional stages that extracts high-level semantic features $F_{\text{enc}} \in \mathbb{R}^{\frac{H}{S} \times \frac{W}{S} \times C_{\text{enc}}}$, where $S \geq 1$ denotes the total output stride (typically $S \in \{8, 16, 32\}$).
- **Decoder (Segmentation Head):** A module that restores spatial resolution to $H \times W$ through bilinear interpolation or transposed convolutions, combining low-level spatial details from earlier encoder layers with abstract high-level features.

4.2 Fully Convolutional Networks (FCN)

Long et al. [8] introduced Fully Convolutional Networks by replacing the final dense (fully connected) classification layers of standard convolutional backbones with convolutional filters.

4.2.1 Fully convolutional transformation

In standard classification networks, the spatial feature map produced by the final convolutional layer is flattened into a vector and passed to dense layers, restricting the network to fixed input resolutions. FCN replaces each dense layer with a 1×1 convolutional layer whose kernel depth matches the dense weight matrix. For a feature map $F \in \mathbb{R}^{h \times w \times C_{\text{in}}}$, a convolutional layer with C filters of size 1×1 computes:

$$Z_{i,j,c} = b_c + \sum_{k=1}^{C_{\text{in}}} W_{c,k} \cdot F_{i,j,k}, \qquad \forall i \in \{1, \ldots, h\},\ j \in \{1, \ldots, w\}, \tag{4.3}$$

producing a coarse score map of dimension $h \times w \times C$. This allows the network to process inputs of arbitrary dimensions in a single forward pass without sliding-window recomputation.

4.2.2 Skip architecture and upsampling

To restore the coarse score map from stride $S = 32$ back to the original image dimensions $H \times W$, FCN employs transposed convolutions (also known as backward strided convolutions or fractionally strided convolutions). A transposed convolution with stride s places the input values on a dilated grid and computes the output via standard convolution, learning an upsampling filter $w \in \mathbb{R}^{K \times K}$:

$$\hat{y}_{i',j'} = \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} w_{u,v} \cdot x_{\lfloor i'/s \rfloor - u,\, \lfloor j'/s \rfloor - v}. \tag{4.4}$$

Upsampling directly from stride 32 produces blurry boundaries. Long et al. therefore designed skip connections:

- **FCN-32s:** Upsamples the final stride-32 score map directly by a factor of 32 in a single step.
- **FCN-16s:** Combines the 2× upsampled stride-32 score map with predictions extracted from the stride-16 stage (e.g., `conv4`), then applies 16× upsampling.
- **FCN-8s:** Combines the 2× upsampled FCN-16s score map with predictions extracted from the stride-8 stage (e.g., `conv3`), followed by an 8× upsampling step.

The implementation used in this work couples an FCN segmentation head with a ResNet-50 backbone.

4.3 DeepLabV3 and atrous convolutions

DeepLabV3, developed by Chen et al. [9], addresses the resolution loss caused by consecutive pooling operations by replacing standard downsampling with atrous (dilated) convolutions and multi-scale pooling modules.

4.3.1 Atrous convolution formulation

Let $x[i, j]$ denote a 2D input feature map and $w[p, q]$ a convolutional filter of size $K \times K$ with $p, q \in \{-M, \ldots, M\}$ where $M = \frac{K-1}{2}$. Standard convolution samples

adjacent positions on the input grid. Atrous convolution introduces a dilation rate $r \in \mathbb{N}^+$, defining the step with which the filter samples the input signal:

$$y[i,j] = \sum_{p=-M}^{M} \sum_{q=-M}^{M} x[i + r \cdot p,\ j + r \cdot q] \cdot w[p,q]. \tag{4.5}$$

When $r = 1$, Eq. (4.5) reduces to the standard convolution. For $r > 1$, the filter samples inputs with spaces of $r - 1$ zeros between consecutive filter taps. The effective filter size K' becomes:

$$K' = K + (K-1)(r-1). \tag{4.6}$$

For example, a 3×3 kernel ($K = 3$) with dilation rate $r = 6$ covers a spatial footprint of $3 + 2 \times 5 = 13 \times 13$ pixels, requiring only 9 learnable parameters. Atrous convolution expands the receptive field without adding parameters or reducing the spatial resolution of the feature maps.

4.3.2 Atrous Spatial Pyramid Pooling (ASPP)

Objects and agricultural corridors appear at varying visual scales depending on camera mounting angle and distance from the vehicle. DeepLabV3 captures contextual information across multiple scales using the Atrous Spatial Pyramid Pooling (ASPP) module.

The ASPP module operates in parallel on the high-level feature map extracted by the backbone (typically at output stride $S = 16$) and consists of five branches:

1. One 1×1 standard convolution with 256 filters and batch normalization.
2. Three 3×3 atrous convolutions with 256 filters each, using dilation rates $r \in \{6, 12, 18\}$ for an output stride of 16 (or $r \in \{12, 24, 36\}$ for stride 8).
3. One Image-Level Pooling branch: to incorporate global image context, this branch computes global average pooling across the spatial dimensions of the input tensor, processes the resulting $1 \times 1 \times C_{\text{in}}$ vector with a 1×1 convolution (256 filters) and batch normalization, and upsamples the output bilinearly to the original ASPP spatial resolution.

The outputs of all five branches are concatenated along the channel dimension, resulting in a tensor with $5 \times 256 = 1280$ channels:

$$F_{\text{concat}} = \left[F_{1\times1},\ F_{\text{atrous},r_1},\ F_{\text{atrous},r_2},\ F_{\text{atrous},r_3},\ F_{\text{image_pool}}\right]. \tag{4.7}$$

Finally, a 1×1 convolution with 256 filters, batch normalization, and dropout (with probability 0.5) projects F_{concat} to 256 channels, followed by a 1×1 classification layer and bilinear upsampling to image resolution $H \times W$.

4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head

While DeepLabV3 with a ResNet-50 backbone provides high segmentation accuracy, its 50.8 million parameters and heavy computational requirement result in high inference latency (86.87 ms per image), exceeding the real-time budget of mobile robot platforms. To achieve real-time operation on edge hardware, Howard et al. [10] developed MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) segmentation head.

4.4.1 Depthwise separable convolutions

The fundamental building block of MobileNet architectures is the depthwise separable convolution, which factors a standard convolution into two separate steps: a spatial depthwise convolution and a channelwise pointwise convolution.

Consider an input feature map $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ and an output feature map $Y \in \mathbb{R}^{H \times W \times C_{\text{out}}}$, processed with spatial filter size $D_K \times D_K$.

- **Standard convolution:** Each filter channel interacts with every input channel simultaneously. The computational cost, measured in Multiply-Accumulate (MAC) operations, is:

$$\text{Cost}_{\text{standard}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2. \tag{4.8}$$

- **Depthwise convolution:** Applies a single $D_K \times D_K$ spatial filter per input channel independently, without channel cross-talk:

$$\text{Cost}_{\text{depthwise}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2. \tag{4.9}$$

- **Pointwise convolution:** Applies C_{out} filters of size 1×1 across all channels to compute linear combinations of the depthwise outputs:

$$\text{Cost}_{\text{pointwise}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.10}$$

The total cost of a depthwise separable convolution is the sum of Eqs. (4.9) and (4.10):

$$\text{Cost}_{\text{sep}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.11}$$

Comparing Eq. (4.11) to Eq. (4.8) yields the computational reduction ratio:

$$\frac{\text{Cost}_{\text{sep}}}{\text{Cost}_{\text{standard}}} = \frac{H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}}{H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2} = \frac{1}{C_{\text{out}}} + \frac{1}{D_K^2}. \tag{4.12}$$

For standard 3×3 filters ($D_K = 3$) and typical channel counts ($C_{\text{out}} \geq 64$), the reduction ratio is approximately $\frac{1}{9} \approx 0.11$. Depthwise separable convolutions thus reduce arithmetic operations and parameter counts by roughly 88–89% relative to standard convolutions.

4.4.2 Inverted residual blocks with linear bottlenecks

In classical residual blocks (such as those in ResNet), the input channels are first compressed via a 1×1 convolution, processed with a 3×3 convolution, and expanded back to the original channel count (narrow $\rightarrow$ wide $\rightarrow$ narrow).

MobileNetV3 reverses this structure using inverted residual blocks (narrow $\rightarrow$ wide $\rightarrow$ narrow):

1. **Expansion layer:** A 1×1 pointwise convolution expands the low-dimensional input C_{in} to a higher-dimensional space $e \cdot C_{\text{in}}$, where $e \in \{3, 6\}$ is the expansion factor.

2. **Depthwise layer:** A 3×3 or 5×5 depthwise separable convolution performs spatial feature extraction in the expanded channel space.

3. **Linear bottleneck:** A 1×1 projection layer compresses the channels back to C_{out}. Crucially, no non-linear activation function is placed after this final projection layer. Because non-linearities such as ReLU discard negative values by projecting them to zero, applying non-linear activations in low-dimensional spaces destroys useful geometric information. Retaining a linear bottleneck preserves manifold topology.

When the input and output channel dimensions match and the stride is 1, a residual shortcut connection adds the input directly to the block output.

4.4.3 Squeeze-and-Excitation (SE) channel attention

To adaptively calibrate channel dependencies, MobileNetV3 incorporates Squeeze-and-Excitation (SE) blocks within the inverted residual structure:

1. **Squeeze:** Global spatial average pooling aggregates the spatial dimensions of feature map $U \in \mathbb{R}^{H' \times W' \times C'}$ into a channel descriptor vector $z \in \mathbb{R}^{C'}$:

$$z_c = \frac{1}{H'W'} \sum_{i=1}^{H'} \sum_{j=1}^{W'} U_{i,j,c}. \tag{4.13}$$

2. **Excitation:** Two fully connected layers parameterized by weights $W_1 \in \mathbb{R}^{\frac{C'}{4} \times C'}$ and $W_2 \in \mathbb{R}^{C' \times \frac{C'}{4}}$ model non-linear channel relationships with a channel reduction ratio of 4:

$$s = \sigma\left(W_2 \cdot \text{ReLU}(W_1 z)\right), \tag{4.14}$$

   where $\sigma(\cdot)$ denotes the hard-sigmoid activation function.

3. **Scale:** The input tensor U is rescaled channel-wise by the activation vector s:

$$\tilde{U}_{i,j,c} = s_c \cdot U_{i,j,c}, \qquad \forall c \in \{1, \ldots, C'\}. \tag{4.15}$$

4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid

The swish non-linearity, defined as $\text{swish}(x) = x \cdot \sigma(x)$, improves gradient flow in deep networks compared to standard ReLU. However, computing the exponential function $\sigma(x) = (1 + e^{-x})^{-1}$ introduces noticeable latency on embedded mobile processors lacking dedicated transcendental hardware units.

MobileNetV3 replaces standard sigmoid and swish with piece-wise linear hard approximations based on $\text{ReLU6}(x) = \min(\max(x, 0), 6)$:

$$\text{h-sigmoid}(x) = \frac{\text{ReLU6}(x+3)}{6}, \tag{4.16}$$

$$\text{h-swish}(x) = x \cdot \text{h-sigmoid}(x) = x \cdot \frac{\text{ReLU6}(x+3)}{6}. \tag{4.17}$$

Eqs. (4.16) and (4.17) eliminate floating-point exponential operations, preserving the performance benefits of swish while minimizing execution time on embedded CPUs and GPUs.

4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture

Standard DeepLabV3 ASPP requires multiple parallel atrous branches with 256 channels each, resulting in significant memory bandwidth requirements. Howard et al. proposed the Lite Reduced ASPP (LR-ASPP) head as a lightweight alternative tailored for mobile segmentation.

The LR-ASPP architecture uses features from two distinct stages of the MobileNetV3-Large backbone:

- A low-resolution, high-semantic feature map $F_{\text{low}} \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times 960}$ extracted at output stride 16.
- A high-resolution, low-semantic feature map $F_{\text{high}} \in \mathbb{R}^{\frac{H}{8} \times \frac{W}{8} \times 40}$ extracted at output stride 8.

The processing steps of LR-ASPP proceed as follows:

1. F_{low} is projected to 128 channels using a 1×1 convolution with batch normalization:
$$V_{\text{low}} = \text{Conv}_{1\times1}(F_{\text{low}}) \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times 128}. \tag{4.18}$$
2. A global context branch computes global average pooling across F_{low} using a large pooling kernel (spatial size matching the feature map), followed by a 1×1 convolution to 128 channels and hard-sigmoid activation:
$$G = \text{h-sigmoid}\left(\text{Conv}_{1\times1}(\text{AvgPool}(F_{\text{low}}))\right) \in \mathbb{R}^{1\times1\times128}. \tag{4.19}$$

3. The low-resolution feature map V_{low} is modulated by the global context gating vector G via element-wise multiplication:

$$M_{\text{low}} = V_{\text{low}} \odot G. \tag{4.20}$$

4. The modulated tensor M_{low} is upsampled by a factor of 2 via bilinear interpolation to match the stride-8 resolution of F_{high}.

5. F_{high} is projected to 128 channels via a 1×1 convolution and added element-wise to the upsampled low-resolution features:

$$F_{\text{fused}} = \text{BilinearUpsample}_{2\times}(M_{\text{low}}) + \text{Conv}_{1\times1}(F_{\text{high}}). \tag{4.21}$$

6. A final 1×1 convolutional classifier projects F_{fused} to C class logits, followed by an $8\times$ bilinear upsampling step back to the original image dimensions $H \times W$.

By discarding multiple parallel dilated convolutions in favor of a single gated global pooling path and direct stride-8 feature addition, LR-ASPP reduces computational operations while retaining global scene context and boundary localization.

4.5 Segment Anything Model 3 (SAM 3) for dataset annotation

Manually annotating agricultural images at pixel precision requires considerable labor: drawing polygons around irregular plant boundaries, inter-row soil, and weed patches requires between 15 and 30 minutes per frame. Foundation models provide a mechanism to scale up annotation through zero-shot visual prompting.

Kirillov et al. [11] established the promptable segmentation paradigm with SAM, using Vision Transformers to generate binary segmentation masks from foreground/background points, bounding boxes, or coarse free-form masks. Carion et al. [5] extended this architecture in SAM 3 by integrating open-vocabulary semantic concepts and text-prompted segmentation.

4.5.1 Architecture overview

SAM 3 consists of three primary components:

1. **Image Encoder:** A heavy Vision Transformer (ViT-H or ViT-L) pre-trained with masked autoencoding and contrastive objectives. The image encoder processes an input image $I \in \mathbb{R}^{1024\times1024\times3}$ and outputs a downsampled $64 \times 64 \times 256$ spatial image embedding. Computing this embedding represents the primary computational cost, but it is executed once per image regardless of the number of subsequent prompts.

2. **Prompt Encoder:** Embeds sparse prompts (spatial coordinate points, bounding boxes) through positional encodings summed with learned prompt-type embeddings. Text prompts are embedded using a frozen contrastive text encoder (such as CLIP or an aligned concept encoder) that maps target category descriptions (e.g., `"lettuce plant"`, `"navigable soil path"`) to the prompt feature space.

3. **Two-Way Mask Decoder:** A lightweight transformer decoder that alternates self-attention among prompt tokens and cross-attention between prompt tokens and the image embedding in both directions. The updated tokens are mapped via linear layers to dynamic filter kernels, which are applied to an upsampled feature map via point-wise dot product to produce candidate segmentation masks and associated IoU confidence scores.

4.5.2 Operational distinction from onboard models

A key architectural distinction must be emphasized between the foundation model and the robotic segmentation network:

- SAM 3 requires over 600 million parameters and multi-gigabyte memory footprints, with single-image forward times exceeding 1000 ms on workstation GPUs. It cannot be deployed directly for closed-loop visual servoing on resource-constrained embedded computers such as the Jetson platform.

- In this thesis, SAM 3 functions strictly as an **offline supervision generator**. Given text prompts representing crop and corridor concepts, SAM 3 generates dense candidate masks across lettuce, tomato, and cucumber image collections.

- The resulting masks are reviewed, filtered, and used as ground-truth targets to train lightweight real-time architectures (FCN, DeepLabV3, and LR-ASPP). The distilled lightweight models execute inference on the robot platform in 10 to 86 ms, maintaining real-time control capability.

4.6 Summary of architectural characteristics

Table 4.1 summarizes the theoretical characteristics of the four architectures analyzed in this chapter, establishing the foundation for the dataset experiments and comparative evaluations presented in Chapter 5.

Table 4.1: Theoretical comparison of deep learning architectures evaluated in this thesis.

Model	Backbone	Context mechanism	Target role	Primary constraint
FCN	ResNet-50	Transposed convolutions and skip connections	Benchmark segmentation	High latency (58.24 ms), coarse boundaries
DeepLabV3	ResNet-50	Parallel Atrous Spatial Pyramid Pooling (ASPP)	High-accuracy benchmark	Highest latency (86.87 ms), 50.8M parameters
LR-ASPP	MobileNetV3-Large	Gated global average pooling and stride-8 fusion	Real-time robotic navigation	Compact capacity, fine texture handling
SAM 3	ViT-Large	Foundation promptable Vision Transformer	Offline automatic annotation	Non-real-time (> 1 s), high GPU memory footprint

Chapter 5

Dataset Preparation and Comparative Model Training

Following the theoretical analysis of deep learning architectures in Chapter 4, this chapter describes the datasets, annotation pipelines, and empirical training comparisons that guide the model configuration for robotic integration.

The experimental study began with an empirical comparison of FCN, DeepLabV3, and LR-ASPP on manually annotated lettuce images. Manual segmentation was time-consuming, so this first comparison was limited to lettuce rather than repeated across every target crop. Its outcome led to the adoption of LR-ASPP for embedded deployment.

SAM 3 was subsequently used to scale up dataset preparation, generating multi-crop training labels across lettuce, tomato, and cucumber collections. The resulting models were evaluated across alternative output label configurations: three classes (plant, path, background), four classes (including covering material), and a binary path-only representation.

This chapter details image acquisition, dataset partitioning, training parameters, and offline benchmark evaluations. The performance of the trained networks is evaluated separately from the annotation quality of the raw SAM 3 masks. Subsequent chapters detail geometric navigation extraction and closed-loop robot validation.

5.1 Image acquisition and class definition

5.1.1 Image sources and acquisition conditions

5.1.2 Semantic classes and labeling targets

Three distinct labeling configurations were evaluated for SAM 3:

- **Four-class configuration:** Segments background, plants, soil, and covering material. This configuration was also used for the lettuce models during architecture selection.
- **Two-foreground-class configuration:** Labels plants and path, with background as the third output class. This produces three total semantic categories.
- **Path-only configuration:** Binary segmentation with background and path as the two output labels. For robotic integration, the path mask is the relevant output. Segmenting soil alone does not identify a unique navigation corridor or establish that it is obstacle-free. The geometric methods in Chapter 6 use the mask to construct a local image-space reference.

5.2 Dataset preparation

5.2.1 Lettuce dataset with manual annotation

The manually annotated lettuce dataset, identified as `Data`, contains 762 image–mask pairs distributed across eight folders. Its masks were prepared from the manual annotations stored in `annotations.json`. The same manual reference is used for the initial architecture comparison and for evaluating the models trained with SAM 3-generated multi-crop labels. Automatically annotated lettuce images belong to the latter training datasets, not to separate lettuce-only training configurations.

5.2.2 SAM 3-assisted annotation for multi-crop datasets

The time required for manual labeling motivated the use of SAM 3 to prepare the multi-crop datasets. Initial use suggested faster mask generation, but the total annotation effort has not been measured in a systematic comparison that includes inspection and correction.

The SAM 3-assisted labeling procedure covers lettuce, tomato, and cucumber images for the multi-crop training configurations. In this workflow, automatic segmentation is used to prepare labels offline; it is distinct from the segmentation model used during navigation.

For the lettuce images annotated with two foreground classes, the reported text prompts are `plants` and `central farm path`. The four-class labeling additionally uses `plastic mulch film` for covering material. Background accounts for the remaining label in each configuration. These reported lettuce prompts do not establish the exact prompts used for every crop in the multi-crop datasets. The binary configuration retains only the path annotation; crop-specific prompts and mask-composition settings for each configuration must be documented from the corresponding labeling runs.

Tomato dataset

The tomato images were labeled using SAM 3 rather than the manual procedure used for the initial lettuce comparison. They contribute to the automatically annotated multi-crop training data. LR-ASPP was adopted after the comparison of the three candidate architectures on lettuce; no separate FCN, DeepLabV3, and LR-ASPP comparison was performed on tomato.

Cucumber dataset

SAM 3 was also used to label the cucumber images, which contribute to the automatically annotated multi-crop training data. As for tomatoes, the architecture choice followed the comparison on manually annotated lettuce, rather than a separate comparison on cucumber.

5.2.3 Dataset partitioning

The reported training runs use an 80% training, 10% validation, and 10% test partition with seed 42. The available `train_model.py` implements these proportions by assigning the integer parts of the training and validation counts and leaving the remaining samples for test. Applied to 762 pairs, this rule gives 609 training, 76 validation, and 77 test samples. These counts describe the intended manual-dataset partition; the saved indices and image lists must establish the actual membership of each evaluated split.

The test set is not used for gradient-based parameter updates. Checkpoint selection uses validation rather than test performance. A common seed alone does not ensure that differently sized or ordered datasets yield identical image partitions. Corresponding image identities must therefore be checked when comparing the manually supervised lettuce model with the automatically supervised multi-crop models, including whether a lettuce evaluation image appears in an automatic model's training set.

The reported operational evaluation selects 42 images from the manual test split belonging to `insalata_grande1`, `insalata_grande2`, and `insalata_media1`.

These folders represent the row conditions selected for the robot application. The 42 images are a subset of the reported 77 test images, not an additional independent test set. Performance on this subset describes those selected conditions, rather than the full range of lettuce images.

5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP

This comparison addresses the architecture choice before changing the annotation procedure or crop coverage. FCN, DeepLabV3, and LR-ASPP were trained and evaluated using the manually annotated lettuce dataset.

5.3.1 Model selection and pretrained weights

FCN, DeepLabV3, and LR-ASPP were chosen from the implementations available in TorchVision. All three were trained on the manually annotated lettuce dataset. Their selection was practical, rather than the outcome of a comparison with every available architecture.

The implementation separates each network into a backbone, which extracts image features, and a segmentation head, which produces class scores at each output location. Transfer learning is implemented by loading pretrained parameters and adapting the output layers to the target labeling task, rather than initializing every network parameter from scratch.

For FCN and DeepLabV3, the selected implementations use a ResNet-50 backbone with pretrained segmentation weights. After these weights are loaded, the final convolution of the main classifier is replaced by a new 1×1 convolution with one output channel per target class.

LR-ASPP is initialized differently. The implementation uses a MobileNetV3-Large backbone and requests its default pretrained weights through the `weights_backbone` argument. However, the `weights` argument of the complete segmentation model is set to `None`. Consequently, the backbone is pretrained, whereas the LR-ASPP segmentation head is newly initialized for the requested number of classes.

This distinction is relevant to the interpretation of the model comparison. The three configurations differ not only in architecture, but also in the extent of their pretrained initialization: FCN and DeepLabV3 retain pretrained segmentation-head layers, whereas LR-ASPP starts with a new segmentation head.

5.3.2 Training configuration

Training is organized into two stages. Before the first epoch, the backbone parameters are excluded from gradient-based updates by setting their `requires_grad` attribute to `False`. The complete main classifier remains trainable, as does the auxiliary classifier when present.

For the first ten epochs, AdamW is constructed using only the trainable parameters, with an initial learning rate of 10^{-3} and weight decay of 10^{-4}. A cosine-annealing scheduler is stepped after each epoch, with a minimum learning rate of 10^{-6}.

At the end of epoch ten, the script enables gradients for all backbone parameters. Fine-tuning of the backbone therefore begins at epoch eleven, together with continued training of the segmentation heads. A new AdamW optimizer is constructed with separate parameter groups: the backbone starts this stage at a learning rate of 10^{-5}, while the main and, where present, auxiliary classifiers start at 10^{-4}. Weight decay remains 10^{-4}.

A checkpoint is saved as `best.pth` whenever the validation loss is lower than all previous values; `last.pth` stores the latest training state. Selection therefore uses minimum validation loss, not maximum test IoU or simply the last epoch.

5.3.3 Evaluation metrics and results

The comparison uses the three CSV records stored in `training insalata`: the FCN, DeepLabV3, and LR-ASPP `*_validation_metrics.csv` exports. Each contains aggregate metrics, the checkpoint epoch, and IoU values for background, plants, soil, and covering material. The reported mIoU agrees with the arithmetic mean of these four class IoUs, including background.

Table 5.1 reports the exported metrics. Inference times have been converted from seconds to milliseconds. The FPS values equal the reciprocal of the mean recorded time; they are not measurements of the complete robot processing or control rate. Checkpoint epochs identify the evaluated weights, not training duration.

Table 5.1: Architecture comparison from the manual lettuce validation exports. Timing values are those recorded in the CSV files, not end-to-end robot latency.

Model	Epoch	mIoU	Path IoU	Mean time (ms)	FPS
FCN	38	0.9232	0.9621	58.24	17.17
DeepLabV3	43	0.9228	0.9651	86.87	11.51
LR-ASPP	194	0.9108	0.9546	10.41	96.07

FCN and DeepLabV3 achieve mIoU values of 0.9232 and 0.9228, respectively,

compared with 0.9108 for LR-ASPP. Using the unrounded CSV values, the LR-ASPP deficit is 1.24 percentage points relative to FCN and 1.19 relative to DeepLabV3. Its mean recorded time is 10.41 ms, compared with 58.24 ms and 86.87 ms. These timings correspond to a 5.60-fold reduction relative to FCN and an 8.35-fold reduction relative to DeepLabV3.

Table 5.2 provides the class-level comparison. LR-ASPP has lower IoU than both alternatives in each class, although its soil/path IoU remains 0.9546. The data support a trade-off between accuracy and latency, not a claim of equal segmentation accuracy.

Table 5.2: Per-class IoU from the same manual lettuce validation exports. Plants, soil, and covering material correspond to `pianta`, `suolo`, and `copertura` in these files.

Model	Background	Plants	Soil/path	Covering material
FCN	0.9464	0.9280	0.9621	0.8565
DeepLabV3	0.9464	0.9277	0.9651	0.8519
LR-ASPP	0.9394	0.9120	0.9546	0.8372

5.3.4 Selection of LR-ASPP

LR-ASPP was adopted because the substantial reduction in recorded inference time justified accepting an mIoU approximately 1.2 percentage points below the other two configurations. It was not the most accurate model, but offered the practical accuracy–latency compromise sought for robot perception. FCN and DeepLabV3 had higher IoU but substantially longer recorded inference times. Subsequent SAM-supervised multi-crop training therefore used LR-ASPP while varying the output label set.

This choice is based on the available offline comparison. The records do not establish a hardware-independent speed ranking or demonstrate that the integrated robot meets its control timing requirements. Those requirements must be assessed using the deployed model and the complete perception and control pipeline.

5.4 Supervision strategy comparison

5.4.1 Training configurations and evaluation targets

Table 5.3 summarizes the four training configurations considered in this comparison. This study is separate from the initial comparison of the three candidate architectures. The manual configuration is trained on lettuce only, whereas all

three SAM-supervised configurations use multi-crop data and differ in their output label sets.

Table 5.3: Training scope and target labels for the comparison of manual and SAM 3-generated supervision. Class counts include background; evaluation uses the manual lettuce reference.

Configuration	Training scope	Output labels
Manual	Lettuce only (`Data`)	Background, plants, path, covering material
SAM-generated, two foreground classes	Multi-crop	Background, plants, path
SAM-generated, four classes	Multi-crop	Background, plants, path, covering material
SAM-generated, binary (`allPlantsSAM`)	Multi-crop	Background, path

Manual masks supply the ground truth for all comparisons. For path-only scoring, the target is the manual soil class and the predicted positive region is the corresponding path output of each model.

5.4.2 Evaluation results

Results to complete after provenance checks: Present the four-class, two-foreground-class, and path-only comparisons on the documented lettuce evaluation set. Report per-class IoU and state which classes each average includes. Add a path-focused summary, identifying the checkpoint and split for each configuration.

5.4.3 Selection of binary path configuration

The binary configuration was selected because its target is directly aligned with path extraction and its observed behavior on the selected operational images supported that choice. The quantitative evidence and limitations are to be completed in this section. This is not a claim that the binary configuration outperforms the manually supervised model or that performance on the selected rows holds across the entire dataset.

5.5 Model export and inference configuration

5.5.1 ONNX export

5.5.2 Inference inputs and outputs

Chapter 6

Geometric Extraction of the Local Navigation Reference

This chapter describes the two geometric extractors implemented in `pf_straight_prior3_single.py` and `pf_straight_prior4_single.py`. They are referred to as Method 3 and Method 4, respectively. Both convert a segmented corridor mask into a local navigation reference by extracting intervals in horizontal strips, associating their boundaries from bottom to top, and fitting the left and right edges separately. The reference is the mean of the two fitted edge lines, rather than a line fitted directly to interval midpoints.

The methods share mask preparation, interval extraction, and residual-based line reweighting. They differ in the support used to select the initial interval, the association cost, the handling of missing observations, and the checks applied before accepting the output. This chapter explains the active implementations and their default parameters; comments that describe inactive or intended behavior are not treated as implemented operations. Each image is processed independently. Temporal filtering, conversion to velocity commands, and closed-loop evaluation belong to the robotic integration stage.

6.1 Input representation and edge-line models

Both functions expect a two-dimensional label mask $M(y, x)$ of height H and width W. Image coordinates increase to the right in x and downward in y. The argument `path_class` identifies the class from which corridor intervals are extracted; its default is 1 in both files. A binary input can therefore use 1 for the target region and 0 elsewhere. For a multiclass segmentation mask, this argument must match the actual label encoding, or the mask must first be converted appropriately. The extractors do not infer the class mapping or use a separate plant class. A

target-region label is not, by itself, a guarantee of traversable space.

The left and right boundaries are represented by

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R. \tag{6.1}$$

Treating x as a function of y permits vertical image lines without an infinite slope. The center reference is

$$x_C(y) = \frac{x_L(y) + x_R(y)}{2} = m_C y + q_C, \qquad m_C = \frac{m_L + m_R}{2}, \quad q_C = \frac{q_L + q_R}{2}. \tag{6.2}$$

Thus, both methods assume locally straight image-space boundaries. The models do not reconstruct terrain or estimate a metric trajectory. Their residuals and width estimates are in pixels, not lateral distances in meters. The two edges need not be parallel: their separation can change linearly with image height.

At each image height, the reference lies halfway between the two predicted boundaries. This construction is not generally equivalent to fitting the observed interval midpoints: the two edge fits can exclude different border-touching samples and assign different residual weights. Keeping the boundaries separate allows each visible side to contribute its own observations, although both sides must have enough samples.

The shared processing stages are:

1. select a lower region of interest and clean the target-class mask;
2. extract and merge projected intervals in horizontal strips;
3. select a seed using the method-specific support and priority rules;
4. associate interval boundaries upward through the image;
5. fit the non-clipped left and right boundaries independently;
6. calculate quality indicators and rasterize the center reference, with method-specific validation and output rules.

6.2 Region of interest and mask preparation

The ROI begins at $y_0 = \lfloor \rho H \rfloor$ and extends to the bottom of the image. The default starting ratio is $\rho = 0.20$ for Method 3 and $\rho = 0.30$ for Method 4. The binary target mask is

$$B(y, x) = \mathbf{1}\{M(y_0 + y, x) = \texttt{path_class}\}, \qquad 0 \le y < H_R = H - y_0. \tag{6.3}$$

Both implementations apply opening with a 5×5 elliptical kernel, then closing with a 9×9 rectangular kernel. Connected components with fewer than 500 pixels are removed; all sufficiently large components are retained, not only the largest one.

Let $\widetilde{B}$ denote the cleaned binary ROI mask. It supplies both interval extraction and local support in the new methods. Unlike the previous midpoint-based implementation, these support calculations do not use the original multiclass mask to compute separate soil and plant ratios. Method 3 additionally consults the original input labels when assessing the rasterized centerline. Method 4 does not perform that centerline-label assessment.

Morphology and interval merging can bridge interruptions or change region connectivity. Consequently, support in $\widetilde{B}$ measures agreement with the processed mask, not independently verified scene geometry. Kernel sizes, component areas, and distance thresholds are in input-mask pixels and depend on image resolution and camera configuration.

6.3 Horizontal strips and candidate intervals

6.3.1 Strip construction and column projection

Both methods use $N = 24$ strips by default, indexed from bottom to top. Their height is $h = \max(1, \lfloor H_R/N \rfloor)$. For strip index s, the ROI-relative bounds are

$$y_s^- = \max(0, H_R - (s+1)h), \qquad y_s^+ = \min(H_R, H_R - sh), \tag{6.4}$$

with the lower image row bound y_s^+ excluded. Empty strips are skipped. The representative full-image coordinate is $y_s = y_0 + (y_s^- + y_s^+)/2$, following the code's bound-based convention. Integer division may leave a remainder above the constructed strips.

For example, with $H_R = 157$ and $N = 24$, the strips cover the lowest $24 \times 6 = 144$ ROI rows. The upper 13 ROI rows do not supply candidates; the ROI and the region actually sampled by the strips are therefore not identical.

Within each strip, the cleaned mask is summed vertically. A column is active whenever its sum exceeds zero, so even one target pixel activates that column. Contiguous active columns form raw intervals. Neighboring intervals are merged when the number of inactive columns between them is at most `max_interval_gap`, whose default is 30 pixels. For a merged interval $[\ell_i, r_i]$, the width is

$$d_i = r_i - \ell_i + 1. \tag{6.5}$$

Intervals narrower than 8 pixels are discarded. There is no fixed maximum interval width in either new method. Instead, an interval touching both image margins

is rejected if the mean of the entire cleaned binary strip is below 0.70. With `border_margin`=2, the flags are

$$t_{L,i} = \mathbf{1}\{\ell_i \leq 2\}, \qquad t_{R,i} = \mathbf{1}\{r_i \geq W - 1 - 2\}. \tag{6.6}$$

These flags indicate possible truncation by the image boundary, rather than verified physical corridor edges.

Column projection merges evidence across different vertical locations; a merged interval can also contain horizontal gaps. Neither its width nor its endpoints prove that its rectangular interior is entirely target class. This matters when interpreting intervals around occlusions or fragmented segmentation.

6.3.2 Method-specific local support

Method 3 computes central support u_i in a band around the rounded interval midpoint. With default half-width 5, the band contains up to 11 columns and spans the strip height. Its value is the mean of $\widetilde{B}$ inside the band, clipped at the image boundaries. Candidates must satisfy $u_i \geq 0.27$ for both seed selection and subsequent association.

This central band checks whether the projected interval contains target pixels near its midpoint, which is useful when merging has joined two regions across a gap. It does not check the whole interval or establish that both endpoints are reliable.

Method 4 instead computes $u_{L,i}$ and $u_{R,i}$ in separate bands centered on ℓ_i and r_i. The effective half-width is $\max(2, \texttt{local_band_half_width})$, equal to 5 by default. These means quantify target-pixel occupancy around each boundary; they are not boundary-detection probabilities. Because a band straddles a boundary, it need not be entirely target class even for a clean interval.

In Method 4, edge support ranks seed candidates and contributes to the final quality score. It is not an acceptance threshold and does not weight the line fit. In particular, `min_edge_support`=0.20 is present in the signature but unused in the active implementation.

6.4 Initialization and bottom-up association

6.4.1 Method 3: lowest supported seed

Method 3 searches the first six constructed strips from the bottom, or all available strips if fewer exist. It selects the lowest strip with at least one candidate satisfying $u_i \geq 0.27$. Within that strip, it minimizes distance between the interval midpoint and $(W-1)/2$; ties favor the widest interval. There is no consecutive-run eligibility test.

Let $\ell_{\text{prev}}, r_{\text{prev}}, d_{\text{prev}}$ denote the last accepted interval. In each strip above the seed, define

$$\delta_i = \max\Big(|\ell_i - \ell_{\text{prev}}|, |r_i - r_{\text{prev}}|\Big), \qquad D_3 = \max(42{,}0.5d_{\text{prev}}). \tag{6.7}$$

Among candidates with adequate central support and $\delta_i \le D_3$, the method minimizes

$$J_{3,i} = \delta_i + 0.1|d_i - d_{\text{prev}}|. \tag{6.8}$$

A strip without an admissible candidate is skipped, leaving the previous interval unchanged. There is no maximum number of consecutive missed strips, and the jump threshold is not increased to account for a longer vertical gap. Strips below the selected seed are not revisited.

6.4.2 Method 4: edge-support seed and gap-limited association

Method 4 pools all candidates in the first six constructed strips. It maximizes $\min(u_{L,i}, u_{R,i})$, then interval width. Unlike Method 3, it does not prioritize proximity to the image center or explicitly choose the lowest eligible strip. Exact ties retain the first candidate in the bottom-up, left-to-right traversal. No minimum-support threshold is applied, so even a weakly supported candidate can initialize tracking.

The boundary displacement δ_i is defined as above, but the jump limit and association cost are

$$D_4 = \max(28{,}0.5d_{\text{prev}}), \qquad J_{4,i} = \delta_i + 0.4|d_i - d_{\text{prev}}|. \tag{6.9}$$

Only the displacement limit determines admissibility; edge support does not enter this cost or an additional tracking gate. Compared with Method 3, the default lower bound on allowable displacement is smaller and width change has a larger cost coefficient. For wide intervals, the width-dependent term can dominate the jump limit in either method.

A counter records consecutive strips without a compatible candidate and resets after each successful association. With `max_gap_strips`=3, tracking stops on the fourth consecutive miss. Stopping does not directly return `lost`: the accumulated points proceed to fitting if at least three are available. The maximum miss count, including the miss that triggers stopping, enters the quality score. The previous interval remains unchanged during gaps.

The stopping rule and gap penalty have different effects: three misses are tolerated for continued association, but already reduce the gap score to zero. A fourth miss stops the search without automatically rejecting a reference supported by the observations collected earlier.

Both methods perform greedy spatial association within one frame, not temporal tracking. Each accepted interval contributes two boundary observations, and at least three accepted intervals are required before the edge-fitting stage.

6.5 Separate robust fitting of the corridor edges

6.5.1 Excluding image-boundary observations

The left fit uses only accepted intervals not flagged as touching the left image margin. The right fit independently excludes intervals flagged as touching the right margin. At least two observations per edge must remain. The two fits can therefore use different strips and span different vertical ranges. A clipped boundary is not replaced by a one-sided centerline estimate: if either edge lacks sufficient samples, both implementations return an empty output with `lost` status.

The retained samples are interval endpoints from the cleaned column projection. They are not points obtained by a dedicated edge detector. Excluding border-touching samples reduces the influence of image truncation, but does not establish the correctness of the remaining boundaries.

6.5.2 Residual-based reweighting

Both methods start each edge fit with uniform weights. There is no semantic, interval-width, or vertical-position base weighting. For an edge sample (x_i, y_i), four reweighting iterations are performed by default, with scale $\sigma = 18$ pixels:

$$e_i^{(t)} = x_i - m^{(t)} y_i - q^{(t)}, \qquad w_i^{(t+1)} = \frac{1}{1 + (e_i^{(t)}/\sigma)^2}. \tag{6.10}$$

Each iteration calls `numpy.polyfit` with degree one and updated weights. Its weights multiply unsquared residuals [12]; the fitted objective at that iteration is

$$(m, q) = \arg\min_{m,q} \sum_i \left[w_i (x_i - m y_i - q) \right]^2 . \tag{6.11}$$

Thus, the effective coefficient of each squared residual is w_i^2, not w_i. The implementation should not be interpreted as a different standard robust-loss estimator solely from the form of its weight rule. Weights are recomputed from the current model rather than accumulated across iterations; no convergence-based stopping criterion is used. Method 3 additionally checks for negligible total weights in its helper.

The final residual for each retained sample is its absolute horizontal deviation from the corresponding fitted edge. Define $\bar{e}$ as the mean over the concatenated left and right residual arrays, and

$$F = \mathrm{clip}(1 - \bar{e}/\sigma, 0, 1). \tag{6.12}$$

Edges with more retained samples contribute more terms to this mean. Reweighting reduces the influence of inconsistent points but cannot correct an erroneous association chain that supports a coherent wrong line. It does not impose parallel edges or estimate genuine path curvature.

6.6 Quality indicators, validation, and image-space output

6.6.1 Method 3: centerline support after rasterization

Method 3 constructs the mean edge line and rasterizes it from the top of the highest accepted strip to the exclusive lower bound of the seed strip. It does not extend the line below an elevated seed. Within this range, it fills rows across skipped strips as well. Predicted horizontal positions are clipped to $[0, W-1]$ and converted to integers; dilation with a 3×3 elliptical kernel thickens the line. Dilation may extend the nonzero output slightly beyond the pre-dilation row range.

Before dilation, the original input labels are checked in a band of half-width 3 around each rasterized point. Let P be the fraction of pixels equal to `path_class` across these clipped bands, and T the number of accepted intervals divided by the number of constructed strips. The score is

$$C_3 = 0.50T + 0.30P + 0.20F. \tag{6.13}$$

The status is `lost` if $C_3 < 0.38$ or $T < 0.25$, `degraded` if neither condition holds but $C_3 < 0.60$, and `ok` otherwise. These checks follow rasterization: a result marked `lost` can still contain a nonempty centerline matrix. No explicit edge-order or positive-width test is applied.

The diagnostic dictionary includes both edge models, the centerline model, row bounds, residual and support indicators, clipped-point counts, and the cleaned full-image ROI mask. Early selection or fitting failures instead return a zero matrix and a reason string, with additional details in some branches.

6.6.2 Method 4: geometric checks and a composite score

Method 4 validates the fitted boundaries before constructing the output. At 16 evenly spaced heights covering the combined range of retained edge samples, it evaluates

$$D(y) = x_R(y) - x_L(y). \tag{6.14}$$

All sampled widths must be strictly positive. The median of these sampled widths is returned as `width`. This check concerns the sampled observation range, not any extrapolation below it.

A separate indicator uses all accepted intervals, including those with border-touching endpoints. For K intervals in association order, let $z_j = r_j - \ell_j$; unlike extraction width d_j, this quantity omits the inclusive-endpoint +1. The width-irregularity indicator is

$$E_D = \frac{Q_{0.90}\left(\{|z_{j+2} - 2z_{j+1} + z_j|\}_{j=1}^{K-2}\right)}{\max(\text{median}_j\,|z_j|,1)}, \qquad A = \text{clip}(1 - E_D/0.30,0,1), \quad (6.15)$$

where $Q_{0.90}$ is the 90th percentile. These are second differences in sample order, not derivatives normalized by vertical distance. If strips are skipped, sample spacing is unequal; even a width varying linearly with image height need not yield zero second differences. The indicator therefore measures an implementation-specific irregularity, not a perspective-invariant physical curvature.

Let S be the mean of the left support values used by the left fit and the right support values used by the right fit, pooled together. With maximum consecutive miss count $g_{\max}$ and allowed gap $g_a = 3$,

$$G = \text{clip}(1-g_{\max}/\max(g_a,1),0,1), \qquad C_4 = 0.25T+0.20S+0.25F+0.20A+0.10G. \quad (6.16)$$

The result is `lost` if the positive-width check fails, $\bar{e} > 2\sigma$, or $C_4 < 0.38$. Otherwise, it is `degraded` for $C_4 < 0.60$ and `ok` for $C_4 \geq 0.60$. There is no separate $T < 0.25$ rejection rule in Method 4.

A result marked `lost` returns a zero output matrix before rasterization, with no centerline model in the post-fit rejection branch. For an accepted result, the mean edge line is rasterized from the top of the highest accepted strip to the bottom of the image, then dilated as in Method 3. This can extrapolate below the seed. It does not check target labels along the rendered centerline, and the edge-order check does not cover the entire rendered range when that range exceeds the sampled observation range.

Diagnostics report the edge and centerline models, mean residual, support, tracking fraction, median fitted width, width irregularity, maximum gap, row bounds, and cleaned ROI mask. Earlier failures return `lost` with reason strings such as `no_seed_candidate`, `tracking_failed`, or `insufficient_edge_points`.

6.6.3 Interpretation of status and confidence

The two confidence values are heuristic scores with different components; they are not calibrated probabilities and should not be compared as if an identical numerical value represented identical reference quality. Method 3 assesses original-mask support along the centerline, whereas Method 4 combines processed-mask edge support with geometric and gap indicators. Neither verifies obstacle-free passage.

A downstream controller must inspect the returned status. In particular, a nonempty Method 3 output does not necessarily denote accepted guidance. Both methods can also clip an out-of-image prediction onto an image border. Neither function specifies a robot stopping or recovery policy. Malformed inputs, unsuitable parameter values, or numerical exceptions are not comprehensively handled by the explicit failure branches.

6.7 Default configurations and comparison

Table 6.1 distinguishes shared defaults from method-specific settings. These are the defaults of the supplied files, not a statement that a controller uses them or that they are experimentally optimal. Overrides and mask encodings must be recorded for each experiment.

Table 6.1: Default configurations of the two edge-based extractors. Distances and areas are expressed in input-mask pixels.

Setting	Method 3	Method 4
Target-class identifier	1	1
ROI starting ratio	0.20	0.30
Number of strips	24	24
Opening / closing kernels	5×5 / 9×9	Same
Minimum component area	500	500
Minimum interval width	8	8
Maximum merged gap	30	30
Image-border margin	2	2
Full-strip occupancy gate	0.70	0.70
Local support-band half-width	5	5
Seed search window	6 strips	6 strips
Central support threshold	0.27	Not used
Minimum edge support threshold	Not used	Inactive parameter
Jump-limit lower bound	42	28
Width-change cost coefficient	0.1	0.4
Consecutive misses allowed	No fixed limit	3
Residual scale / iterations	18 / 4	18 / 4
Centerline assessment half-width	3	Not used
Width-error score scale	Not used	0.30
Lost / acceptable score thresholds	0.38 / 0.60	0.38 / 0.60
Minimum tracking fraction gate	0.25	Not used
Output dilation kernel	3×3	3×3

The principal difference is not the line model but the selection and validation

surrounding it. Method 3 favors a supported interval near the image center in the lowest eligible strip, accepts arbitrarily long association gaps, and evaluates the rendered line against the original mask. Method 4 favors joint support of the two boundaries across the bottom search window, limits consecutive gaps, penalizes width changes more strongly, and rejects outputs failing its geometric or score checks. These choices can affect different failure cases, but they do not establish that one method is more accurate or faster without evaluation.

Both methods can merge separated regions, follow an incorrect but consistent interval chain, or infer a line through an unobserved gap. A wide or well-supported interval is not necessarily the intended row. The two-line representation retains boundary information separately, but requires sufficient non-clipped samples on both sides. Independent edge fits may be supported over different height ranges, and their mean can include extrapolation of one or both boundaries.

Evaluation should separately assess interval association, edge fitting, centerline accuracy, rejection behavior, and closed-loop motion. A controlled comparison should use the same input masks and first align shared settings such as ROI and strip count, so differences are not attributed solely to algorithmic rules when input regions also differ. Useful component comparisons include seed priority, width-change cost, gap stopping, and post-fit validation. The descriptions here specify implemented behavior, not results of those comparisons.

6.8 Effect of strip height and number

The strip count controls vertical sampling and the maximum number of interval observations. More strips provide thinner vertical regions and potentially more points per boundary fit. Taller strips aggregate more mask pixels but mix evidence from different image heights, and column projection can conceal interruptions. Increased strip height does not guarantee more reliable boundary estimates or a numerically better fit.

For a 224×224 input, Method 3 has $y_0 = 44$ and $H_R = 180$; Method 4 has $y_0 = 67$ and $H_R = 157$. Table 6.2 uses the exact integer-division rule for these illustrative dimensions, rather than assuming the same ROI height for both methods. At $N = 24$, the strip heights are 7 and 6 pixels, respectively. A fit can use fewer than N points because of seed location, missed strips, early stopping, or border-point exclusion; the two edges can have different sample counts.

Changing N also changes the vertical extent of a fixed six-strip seed window and, in Method 4, the extent of a three-strip tolerated gap. These settings should therefore be considered together. The width-irregularity indicator depends on the sampling sequence, while tracking fractions use the number of constructed strips as their denominator. Selecting N requires evaluating boundary and centerline errors,

Table 6.2: Strip heights for an illustrative 224×224 mask and the default ROI ratios. The point count is an upper bound per edge, not a guaranteed number of fitted samples.

Strips N	Method 3 height	Method 4 height	Max. edge samples
24	7 px	6 px	24
20	9 px	7 px	20
16	11 px	9 px	16
12	15 px	13 px	12
8	22 px	19 px	8

accepted-point counts, rejection rates, and processing time under the intended image resolution and field conditions. No optimal strip count is inferred from the implementation alone.

Chapter 7

Robot Platform and Integration

This chapter describes the hardware and software components of the robot platform used for field validation. It covers the camera configuration, computing hardware, software architecture using ROS, and the deployment of the trained segmentation model for real-time inference.

7.1 Robot hardware and sensors

7.1.1 Robot platform

7.1.2 Camera configuration

7.1.3 Computing platform

7.2 Software architecture

7.2.1 ROS integration

The software pipeline is implemented using the Robot Operating System (ROS). ROS provides a modular architecture where perception, geometric extraction, and control components communicate through topics and services.

Perception node

The perception node receives camera images, runs the segmentation model, and publishes the path mask. Input images are preprocessed according to the training configuration before inference.

Geometric extraction node

The geometric extraction node subscribes to the segmentation mask, applies Method 3 or Method 4 from Chapter 6, and publishes the navigation reference as a line or position message.

Control node

The control node converts the navigation reference into velocity commands using proportional control based on lateral displacement and heading error.

7.2.2 Perception pipeline implementation

7.2.3 Control interface

7.3 Deployment configuration

7.3.1 Model export to ONNX

The trained LR-ASPP model is exported to ONNX format for deployment. ONNX Runtime provides optimized inference across different hardware platforms. The export procedure preserves model weights and input/output specifications from the training checkpoint.

7.3.2 Real-time inference

The deployed model processes camera frames at the rate required for closed-loop control. Input preprocessing matches the training transformations. The segmentation mask is decoded and passed to the geometric extraction stage.

7.3.3 Timing and latency

End-to-end latency includes image acquisition, preprocessing, segmentation inference, geometric extraction, and control computation. The total pipeline latency must remain below the control update period to maintain stable navigation.

Chapter 8

Experimental Validation

This chapter reports the experimental evaluation of the segmentation models, geometric extractors, and integrated navigation system. It covers offline evaluation using recorded images and, if completed, closed-loop field trials.

8.1 Offline evaluation protocol

8.1.1 Segmentation metrics

Segmentation performance is measured using intersection over union (IoU) for each semantic class. For a class c, IoU is defined as:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{8.1}$$

where true positives, false positives, and false negatives are computed against the manual reference masks.

Mean IoU (mIoU) averages the per-class IoU values. For the four-class configuration, mIoU includes background, plants, path, and covering material. Path IoU is the task-specific metric most relevant for navigation.

Architecture comparison results

The architecture comparison results reported in Section 5.3.3 demonstrate that LR-ASPP achieves path IoU of 0.9546 with inference time of 10.41 ms, compared to FCN (0.9621, 58.24 ms) and DeepLabV3 (0.9651, 86.87 ms). The 1.2 percentage point mIoU deficit is offset by a 5–8× reduction in inference latency.

Supervision strategy results

8.1.2 Geometric reference accuracy

Geometric reference quality is assessed by comparing Method 3 and Method 4 outputs on the same evaluation masks. Metrics include:

- Reference line parameters (slope and intercept)
- Lateral deviation from manual reference (if available)
- Rejection rate (percentage of images where extraction fails validation)
- Computational time

8.2 Field validation

8.2.1 Test conditions

8.2.2 Closed-loop navigation results

8.2.3 Qualitative observations

8.3 Discussion

8.3.1 Segmentation performance

The architecture comparison demonstrates that LR-ASPP provides a practical accuracy–speed trade-off for robotic deployment. Its path IoU of 0.9546 on manually annotated lettuce is sufficient for geometric extraction, while its inference time supports real-time operation.

The supervision strategy comparison addresses the effort required for dataset preparation. SAM 3-assisted annotation reduces manual labeling time, but its quality and the need for inspection remain to be quantified systematically.

8.3.2 Geometric extraction performance

Method 3 and Method 4 differ in their association strategy and validation checks. Their relative performance depends on mask quality, row structure, and parameter tuning.

8.3.3 Limitations

The experiments are subject to several limitations:

- Segmentation evaluation uses manually annotated lettuce as ground truth. Performance on other crops is inferred from multi-crop training but not directly validated against manual references for tomato or cucumber.
- Geometric extractors assume locally straight boundaries. They do not handle curves or complex row patterns.
- Offline evaluation does not establish closed-loop navigation performance. Segmentation IoU and geometric reference quality are necessary but not sufficient for successful field operation.
- If field trials were not completed, the integrated system's behavior under real navigation conditions remains unvalidated.

Chapter 9

Conclusions and Future Work

This chapter summarizes the main findings, discusses the contributions and limitations of the work, and identifies directions for future research.

9.1 Main contributions

This thesis investigated vision-based local guidance for agricultural row navigation through segmentation model selection, dataset preparation with model-assisted annotation, and geometric reference extraction. The main contributions are:

- **Architecture comparison for robotic segmentation:** FCN, DeepLabV3, and LR-ASPP were compared on manually annotated lettuce images. LR-ASPP was selected for its practical accuracy–speed trade-off, achieving path IoU of 0.9546 with 10.41 ms inference time compared to 58.24 ms and 86.87 ms for the other architectures. This 5–8× latency reduction supports real-time operation despite a 1.2 percentage point mIoU deficit.

- **SAM 3-assisted dataset preparation:** Multi-crop training datasets for lettuce, tomato, and cucumber were prepared using SAM 3 text-prompted segmentation. This reduced manual annotation effort, although systematic measurements of time savings and inspection requirements remain to be completed.

- **Evaluation of output label configurations:** Models with four-class, three-class, and binary path outputs were compared. The binary configuration aligns directly with the navigation task and was selected for robotic integration.

- **Edge-based geometric extractors:** Two methods for converting corridor masks into image-space navigation references were implemented. Both use horizontal strip extraction and independent fitting of left and right boundaries, differing in seed selection, association strategy, and validation checks.

9.2 Conclusions on segmentation and annotation

The architecture comparison establishes that LR-ASPP is suitable for real-time agricultural navigation when inference speed is a constraint. Its segmentation accuracy on lettuce is sufficient for path extraction, and its reduced computational cost makes it deployable on robot hardware.

SAM 3-assisted annotation demonstrates that foundation models can reduce manual labeling effort for agricultural datasets. The quality of automatically generated masks and their effect on trained model performance depend on crop appearance, text prompt formulation, and the need for manual inspection. Future work should measure annotation time systematically, including review and correction, to quantify the actual savings.

The choice of output label configuration affects both annotation effort and downstream geometric extraction. Binary path segmentation focuses the model directly on the navigation target, simplifying both the labeling task and the geometric stage. This comes at the cost of discarding plant and covering-material information that might be useful for other tasks.

9.3 Conclusions on geometric extraction

The geometric extractors convert segmentation masks into navigation references by selecting intervals from horizontal strips and fitting corridor boundaries independently. This approach shares the local-evidence structure of adaptive multi-ROI methods while using a simpler straight-line model.

Method 3 and Method 4 differ in their association strategy and validation. Their relative performance depends on mask quality, row regularity, and parameter tuning. Both methods assume locally straight boundaries and do not handle curves or complex patterns. Validation checks reject outputs that fail quality thresholds, but the controller must handle cases where no valid reference is available.

9.4 Limitations

The work is subject to several limitations:

- **Evaluation scope:** Segmentation models were evaluated on manually annotated lettuce. Multi-crop training includes tomato and cucumber, but their segmentation performance is not validated against manual references for those crops.
- **Annotation quality measurement:** The time required for SAM 3-assisted annotation has not been measured systematically. The comparison does not quantify inspection effort or the frequency of manual corrections.
- **Geometric model assumptions:** The extractors assume locally straight corridor boundaries. They do not handle curves, gaps, or complex row patterns that violate this assumption.
- **Field validation:** If closed-loop field trials were not completed, the integrated system's navigation performance under real operating conditions remains unvalidated. Offline segmentation IoU and geometric reference accuracy do not establish successful row following.

9.5 Future work

Several directions for future research emerge from this work:

9.5.1 Dataset and annotation

- Measure annotation time systematically for manual and SAM 3-assisted workflows, including inspection and correction.
- Prepare manual ground-truth masks for tomato and cucumber to validate multi-crop model performance directly.
- Investigate active learning strategies to select images for manual annotation that improve model generalization most effectively.
- Explore other foundation models or fine-tuned SAM variants specialized for agricultural segmentation.

9.5.2 Segmentation and model deployment

- Evaluate additional lightweight architectures (e.g., BiSeNet, FastSCNN) for further latency reduction.
- Investigate quantization and model compression techniques to enable deployment on edge devices.

- Test temporal consistency methods that use previous frames to improve segmentation and reduce flickering.
- Explore uncertainty estimation to detect when segmentation confidence is low and fallback strategies are needed.

9.5.3 Geometric extraction

- Extend the geometric model to handle curved paths using splines or piecewise linear segments.
- Investigate robust estimation techniques (e.g., RANSAC, M-estimators) for association and fitting under severe occlusions.
- Develop failure detection and recovery strategies when no valid reference can be extracted.
- Compare strip-based methods with alternative geometric representations (e.g., histogram methods, crop-edge detection).

9.5.4 System integration and field validation

- Complete closed-loop field trials to validate end-to-end navigation performance under variable lighting, growth stages, and row conditions.
- Measure lateral tracking error and success rate over extended distances and multiple crop types.
- Integrate temporal filtering and prediction to handle brief reference loss or occlusions.
- Investigate fusion with other sensors (e.g., depth, IMU) to improve robustness.
- Develop a complete autonomy system including global path planning, obstacle avoidance, and task-specific behaviors.

9.6 Closing remarks

This thesis addressed the segmentation, dataset preparation, and geometric extraction stages of vision-based agricultural navigation. The selection of LR-ASPP balances accuracy and speed for robotic deployment. SAM 3-assisted annotation reduces manual effort, and the geometric extractors convert masks into navigation references using horizontal strip selection and independent edge fitting.

The work establishes a foundation for camera-based local guidance in structured crop environments. Future research should complete field validation, extend the geometric model to handle curves and gaps, and systematically measure annotation savings to guide dataset preparation strategies.

Bibliography

[1] Enrico Mendez, Javier Piña Camacho, Jesús Arturo Escobedo Cabello, and Alfonso Gómez-Espinosa. «Autonomous Navigation and Crop Row Detection in Vineyards Using Machine Vision with 2D Camera». In: *Automation* 4.4 (Sept. 2023), pp. 309–326. DOI: 10.3390/automation4040018 (cit. on pp. 6, 9–11).

[2] Johannes Kneip, Patrick Fleischmann, and Karsten Berns. «Crop Edge Detection Based on Stereo Vision». In: *Robotics and Autonomous Systems* 123, 103323 (Jan. 2020). DOI: 10.1016/j.robot.2019.103323 (cit. on pp. 6, 8, 10, 11).

[3] Rajitha de Silva, Grzegorz Cielniak, Gang Wang, and Junfeng Gao. «Deep Learning-Based Crop Row Detection for Infield Navigation of Agri-Robots». In: *Journal of Field Robotics* 41.7 (2024), pp. 2299–2321. DOI: 10.1002/rob.22238 (cit. on pp. 6, 7, 9–11).

[4] Alessandro Navone, Mauro Martini, Marco Ambrosio, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «GPS-Free Autonomous Navigation in Cluttered Tree Rows with Deep Semantic Segmentation». In: *Robotics and Autonomous Systems* 183, 104854 (Jan. 2025). DOI: 10.1016/j.robot.2024.104854 (cit. on pp. 6, 7, 9–11).

[5] Nicolas Carion et al. «SAM 3: Segment Anything with Concepts». In: *International Conference on Learning Representations (ICLR)*. 2026. arXiv: 2511.16719 [cs.CV]. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/e0982cbc81401df3430ee1ff780dc7a2-Abstract-Conference.html (cit. on pp. 8, 20).

[6] Jiqing Chen, Hu Qiang, Jiahua Wu, Guanwen Xu, and Zhikui Wang. «Navigation Path Extraction for Greenhouse Cucumber-Picking Robots Using the Prediction-Point Hough Transform». In: *Computers and Electronics in Agriculture* 180, 105911 (Jan. 2021). DOI: 10.1016/j.compag.2020.105911 (cit. on pp. 8, 10, 11).

[7] Xia Li, Junhao Su, Zhenchao Yue, and Fangtao Duan. «Adaptive Multi-ROI Agricultural Robot Navigation Line Extraction Based on Image Semantic Segmentation». In: *Sensors* 22.20, 7707 (Oct. 2022). DOI: 10.3390/s22207707 (cit. on pp. 9, 10).

[8] Jonathan Long, Evan Shelhamer, and Trevor Darrell. «Fully Convolutional Networks for Semantic Segmentation». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 3431–3440. DOI: 10.1109/CVPR.2015.7298965 (cit. on p. 14).

[9] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. «Rethinking Atrous Convolution for Semantic Image Segmentation». In: *arXiv preprint arXiv:1706.05587* (2017) (cit. on p. 15).

[10] Andrew Howard et al. «Searching for MobileNetV3». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 1314–1324. DOI: 10.1109/ICCV.2019.00140 (cit. on p. 17).

[11] Alexander Kirillov et al. «Segment Anything». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2023, pp. 4015–4026. arXiv: 2304.02643 [cs.CV] (cit. on p. 20).

[12] NumPy Developers. *numpy.polyfit: NumPy v2.4 Manual*. URL: https://numpy.org/doc/2.4/reference/generated/numpy.polyfit.html (visited on 09/13/2026) (cit. on p. 36).